

Assignment 3

Fleet Management

1. Introduction

This project implements a **Fleet Highway Simulator** using Java Multithreading and Swing GUI. It simulates multiple vehicles running concurrently to update a shared highway distance counter. The system provides real-time user controls and demonstrates a **Race Condition** with a toggleable synchronization fix to ensure data consistency.

2. Steps to compile, run and test the program

- Compiling the code:- Open your terminal/command prompt in the 'src' directory and run: `javac com/fleet/vehicles/*.java com/fleet/interfaces/*.java com/fleet/exceptions/*.java com/fleet/simulator/*.java`.
- Run the Simulator:- `java com.fleet.simulator.FleetSimulatorGUI`

3. Overview of the design and GUI layout

1. Architecture:-

- Highway.java (Shared Resource): Maintains the 'totalDistance' counter. It contains a toggleable 'synchronized' mode to simulate or fix race conditions.
- VehicleThread.java (Worker Thread): Implements Runnable. Each vehicle runs in its own thread, updating its state (mileage, fuel) and the shared Highway resource every second.
- FleetSimulatorGUI.java (Controller/View): The main Swing application that manages the threads and visualizes data.

2. GUI Layout:-

- Top Panel: Contains the Application Title and Global Controls (Start, Pause, Resume, Stop buttons) and the "Race Condition Fix" checkbox.
- Center Scroll Pane: Displays a list of "Vehicle Cards". Each card visualizes a specific vehicle's status (Running/Paused/Stopped), Fuel Level (ProgressBar), and Mileage.

- Bottom Panel: Displays the shared "Total Fleet Distance" counter.

3. Thread Control via GUI:

- Start: Initializes and starts a new thread for every vehicle in the list.
- Pause/Resume: Sets a volatile boolean flag ('paused') inside each VehicleThread. The run() loop checks this flag to temporarily sleep without terminating.
- Stop: Sets a volatile boolean flag ('running') to false, causing the run() loop to terminate gracefully.

4. Explanation of how the simulation threads were controlled via the GUI

The GUI manages the VehicleThread workers using **volatile boolean flags** to ensure immediate visibility across threads:

- **Start:** Initializes and launches a new Thread for each vehicle, triggering the run() loop.
- **Pause:** Sets a paused flag to true. The thread loop detects this and sleeps continuously until cleared, preserving state.
- **Resume:** Sets the paused flag to false, allowing the loop to resume movement logic immediately.
- **Stop:** Sets a running flag to false, causing the while loop to break and the thread to terminate.

5. Description of the race condition and synchronisation fix

A Race Condition occurs in this simulation when multiple 'VehicleThread' instances attempt to read, increment, and write to the shared 'Highway.totalDistance' variable simultaneously.

- Demonstration (The Bug):

In 'Highway.addDistance()', when synchronization is disabled, the code performs the addition in three distinct steps:

1. Read current total into a temp variable.
2. Sleep for 1ms (Artificial delay to force context switching).
3. Write (temp + distance) back to total.

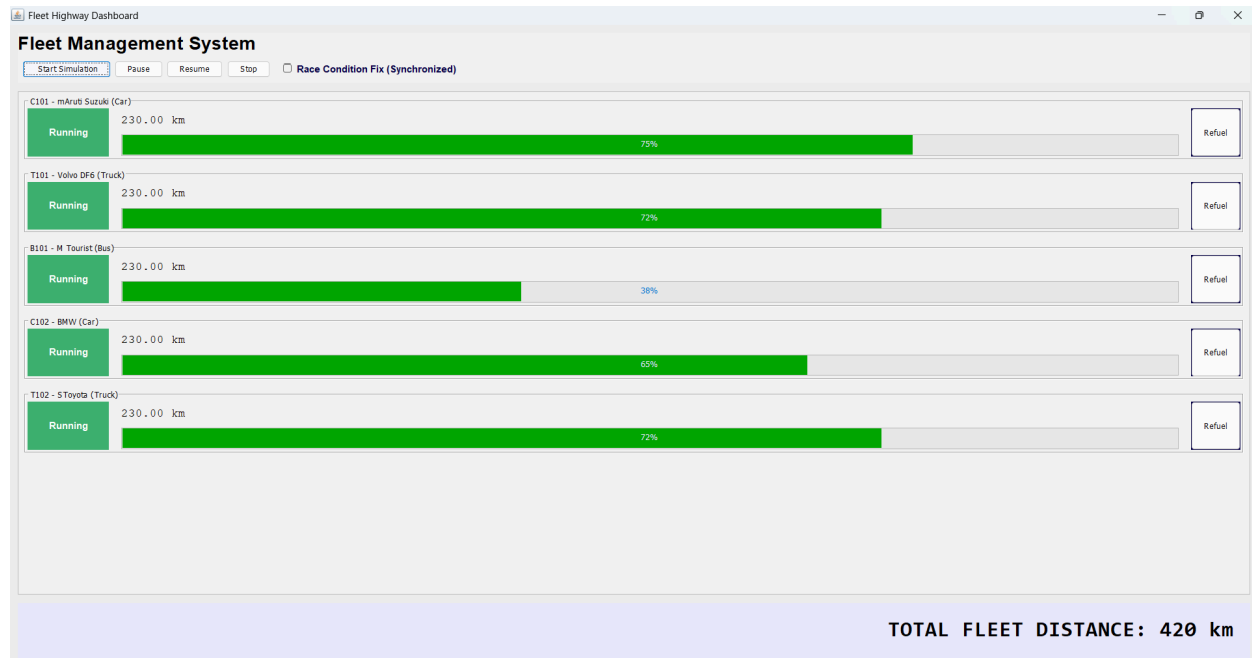
Without a lock, Thread A and Thread B can both read the same starting value (e.g., 100). Both add 10, and both write 110. One update is lost effectively.

- The Fix (Synchronization):

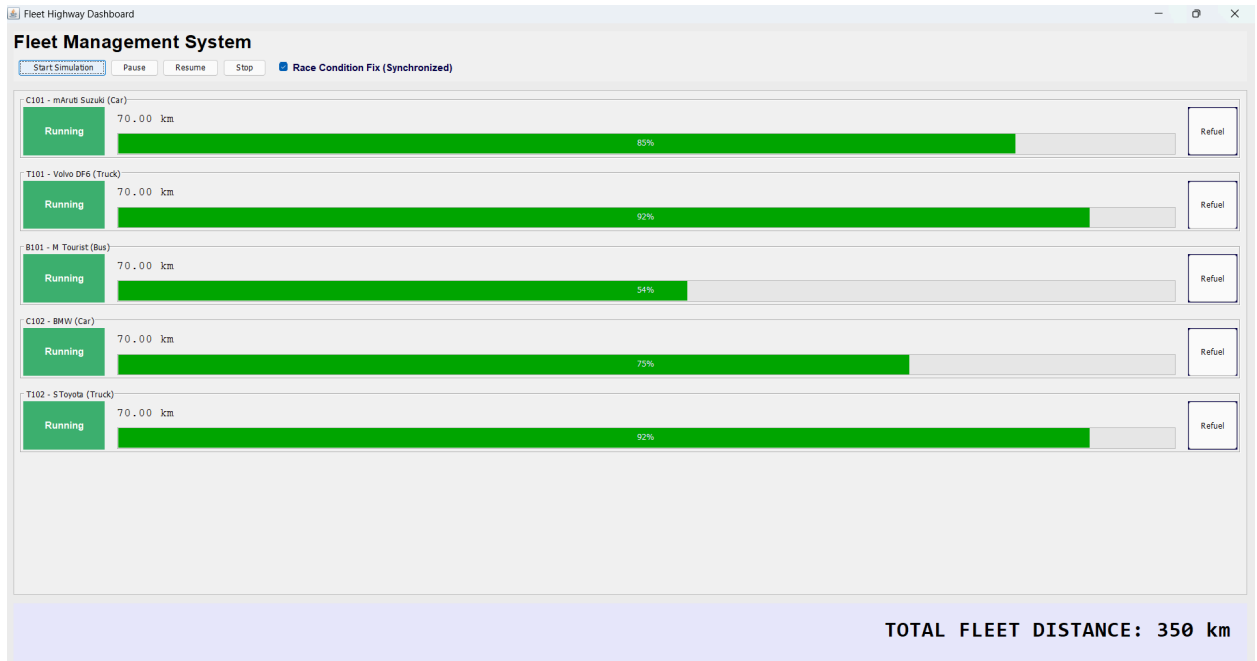
When the "Race Condition Fix" checkbox is selected, the critical section is wrapped in a 'synchronized(this)' block. This acts as a Monitor Lock, ensuring that only one thread can execute the read-modify-write sequence at a time, guaranteeing data consistency.

- **After a couple of seconds, when running, disabling the race condition fixes the checkbox.**

The total distance is wrong especially when the simulator is paused and resumed or we can say when we interfere with it.



- **After a couple of seconds, when running, enabling the race condition, fix the checkbox.**



The bug is fixed as of now.

5. GUI thread-safety considerations

Java Swing components are not thread-safe and must only be modified from the Event Dispatch Thread (EDT). Since our 'VehicleThread' logic runs in background worker threads, we cannot directly call methods like 'label.setText()' or 'progressBar.setValue()'. Doing so could cause graphical glitches or application crashes.

To solve this, all UI updates inside 'VehicleThread.java' are wrapped in:

```
SwingUtilities.invokeLater() -> { ... };
```